SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER APPLICATIONS

TIRUCHIRAPPALLI

RECORD WORK

PCA25C04J – DATA ANALYSIS UNSING PYTHON PROGRAMMING

Register No. :

Name of the Student :

Semester / Year : **II / I**

Programme : **M.C.A**

April - 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER APPLICATIONS

REGISTER NO._____

BONAFIDE CERTIFICATE

Certified as the bonafide record of work done by RA25322410500 Register Number 2 of 1 (Semester/Year), MASTER OF COMPUTER APPLICATIONS **Programme** in the practical course **PCA25C04J – DATA ANALYSIS USING PYTHON PROGRAMMING** at **SRM Institute of Science and Technology, Tiruchirappalli Campus** during the Academic Year, 2025-2026.

Faculty In-charge **Head of the Department**

Submitted for the End Semester Examination held on _____

Examiner-1 **Examiner-2**

INDEX

S.NO	DATE	NAME OF THE PROGRAM	PAGE NO	SIGNATURE
1.		Python Simple Programs	1	
1.a		Implementation of Recursion in Python - Factorial	1	
1.b		Python Program to Check Prime Number	3	
1.c		Arithmetic Operators in Python	5	
1.d		Python Program for Divisible Numbers By 2 And 3	7	
1.e		Python Program to Compute the GCD of Two Numbers	9	
1.f		Strings in Python	11	
2.a		Lists in Python	14	
2.b		Tuples in Python	18	
2.c		Dictionaries in Python	21	
3.		Illustration On Lambda and Filters in Python	24	
4.		Implementing Inheritance	26	
5.		Implementing Method Overloading	29	
6.		Illustration On How to Raise an Exception	31	
7.		Implementing Modules - [Creating and Importing]	33	

8.		Implementing Threads	36	
9.a		Illustration on Command Line Arguments Using Python	39	
9.b		Illustration On Regular Expressions in Python	41	
10.		Descriptive Statistics Using Numpy	43	
11.		Illustrate Indexing Operations in Data Frames	46	
12.		Illustrate Various Plots Using Pandas and Matplotlib	48	
13.		Building Gui Applications with Tkinter	56	
14.		Creating Tables Using SQLite	58	
15.		Illustration On Database Connectivity	62	

Program No: 1 A File Name: Ex. No: Date: _____	IMPLEMENTATION OF RECURSION IN PYTHON (FACTORIAL)

AIM:

Write a Python program to find factorial calculation using recursion.

ALGORITHM:

Step 1: Get a positive integer input (n) from the user.

Step 2: 0! = 1 & 1! = 1

Step 3: n! = n x (n-1)!

Step 4: The factorial of a negative integer is undefined (i.e., (-n)! is not defined).

Step 5: The factorial function grows very quickly, and for large values of n, it can overflow the capacity of standard data types like integers.

Step 6: Iterate from 1 to n using a for loop (for loop is used to increment the number up to the given input)

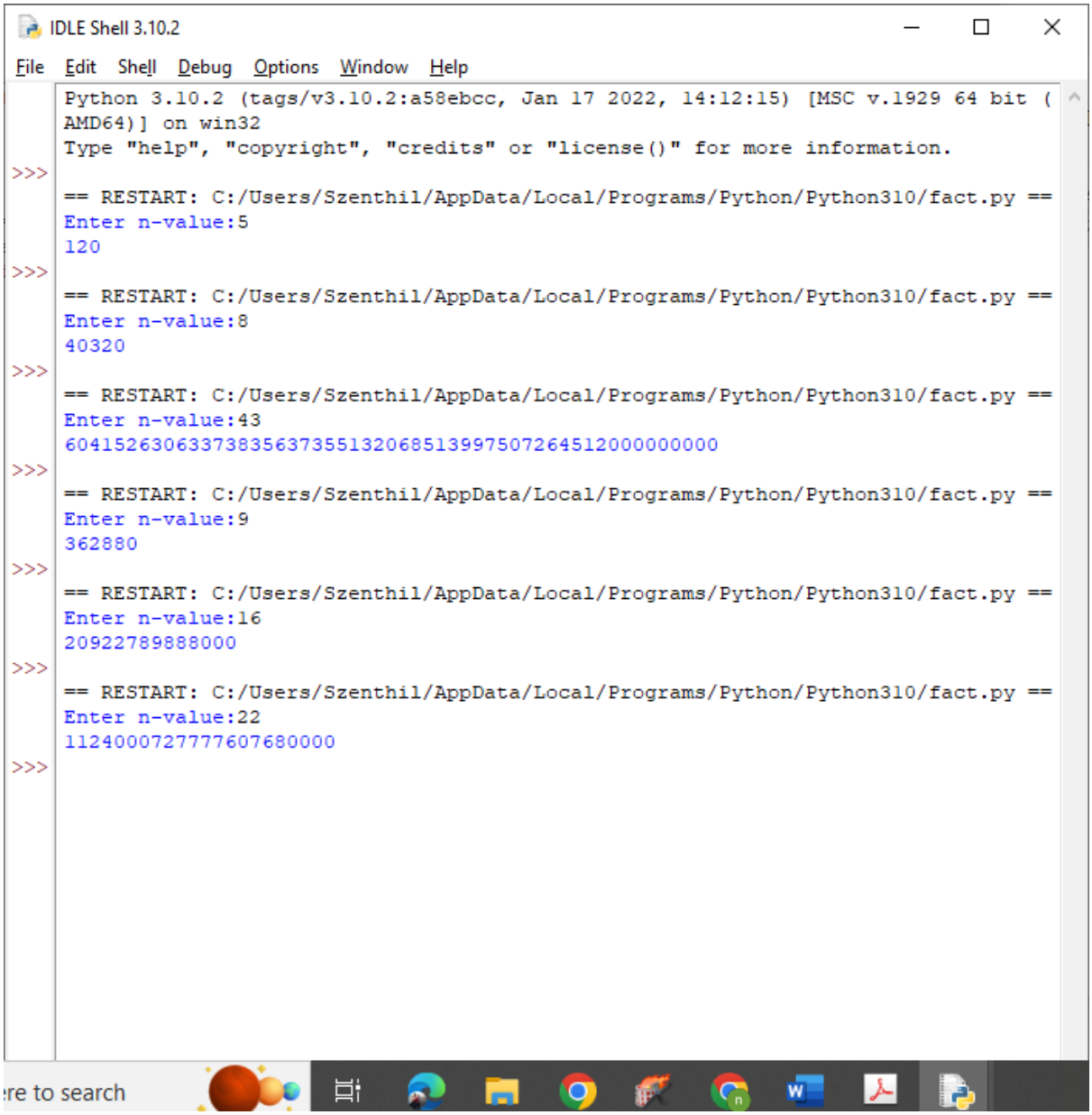
Step 7: Using the below formula, calculate the factorial of a number = f*i.

Step 8: Print the output i.e., the calculated factorial.

PROGRAM:

```
def factorial(n):
    if (n==0 or n==1):
        return 1
    else:
        return n * factorial(n-1)
n = int(input('Enter n-value:'))
res = factorial(n)
print(res)
```

OUTPUT:



```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:5
120
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:8
40320
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:43
604152630633738356373551320685139975072645120000000000
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:9
362880
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:16
20922789888000
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/fact.py ==
Enter n-value:22
1124000727777607680000
>>>
```

RESULT:

Thus, the python program for computing factorial is successfully completed.

Program No: 1 B File Name: Ex. No: Date: _____	PYTHON PROGRAM TO CHECK PRIME NUMBER

AIM:

To write a Python Program for to check whether the given number is Prime number or not.

ALGORITHM:

Step 1: Initialize a for loop starting from 2 and ending at the number

Step 2: Check if the number is divisible by 2

Step 3: Repeat till the number -1 is checked for

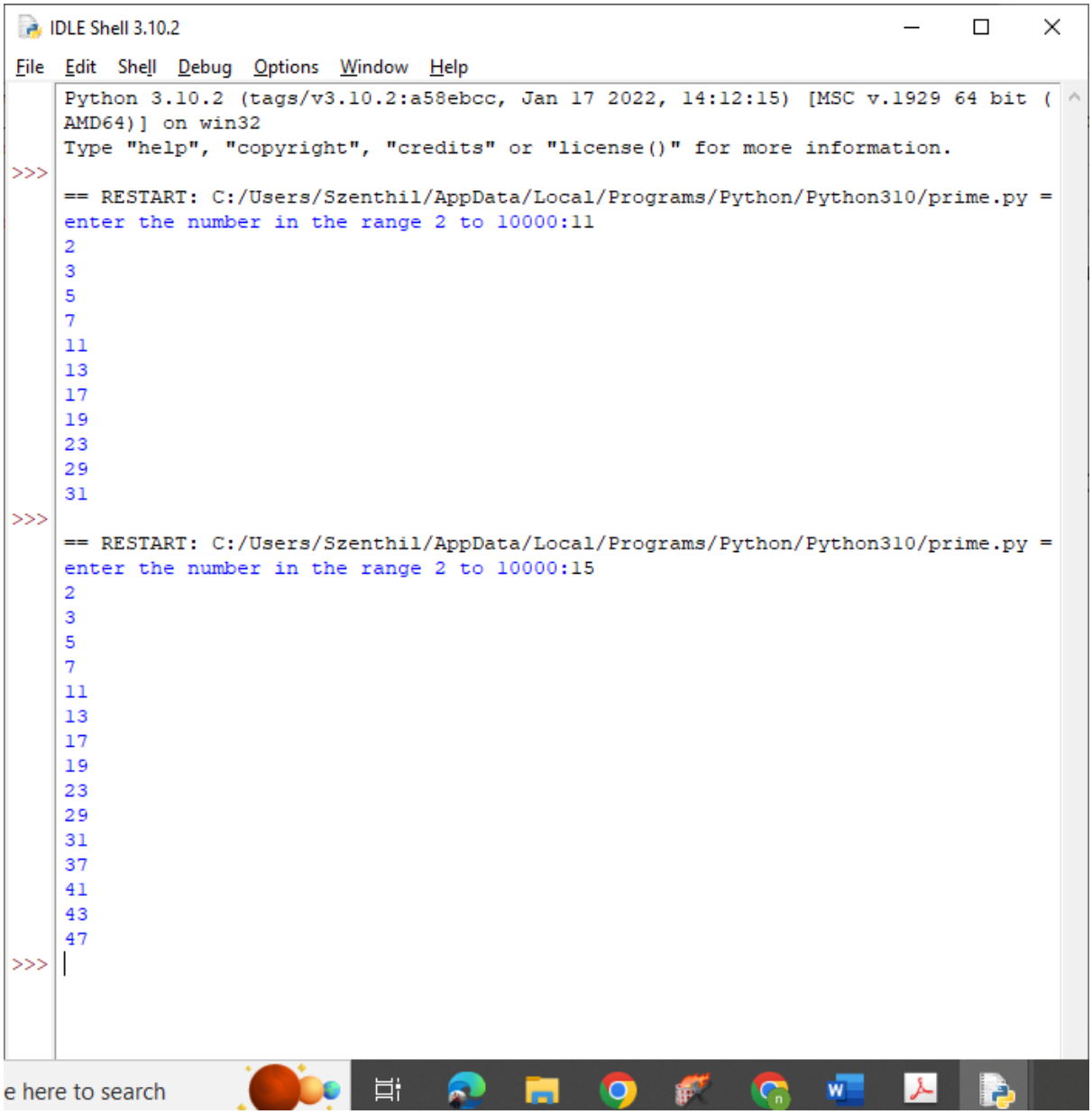
Step 4: In case, the number is divisible by any of the numbers, the number is not prime

Step 5: Else, it is a prime number

PROGRAM:

```
number=int(input("enter the number in the range 2 to 10000:"))
pNumbers=[]
for num in range(2,10000):
    if all(num%i!=0 for i in range(2,num)):
        pNumbers.append(num)
for i in range(number):
    print(pNumbers[i])
```

OUTPUT:



RESULT:

Thus, the python program for finding the prime number is successfully completed.

Program No: 1 C File Name: Ex. No: Date: _____	ARITHMETIC OPERATORS IN PYTHON

AIM:

Write a Python program using Arithmetic Operators to perform mathematical operations like addition, subtraction, multiplication, division, etc.

ALGORITHM:

Step 1: Get two integers as inputs: a & b.

Step 2: Give the input value for a.

Step 3: Give the input value for b.

Step 4: Print the (Sum, Subtraction, Multiplication, Division, Floor Division, Modulo and Power)

PROGRAM:

```
a = int(input("Enter the First Number: "))

b = int(input("Enter the Second Number: "))

print ('Sum: ', a + b)                # addition

print ('Subtraction: ', a - b)         # subtraction

print ('Multiplication: ', a * b)      # multiplication

print ('Division: ', a / b)            # division

print ('Floor Division: ', a // b)     # floor division

print ('Modulo: ', a % b)              # modulo

print ('Power: ', a ** b)              # a to the power b
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/operators.py
Enter the First Number: 15
Enter the Second Number: 23
Sum:  38
Subtraction:  -8
Multiplication:  345
Division:  0.6521739130434783
Floor Division:  0
Modulo:  15
Power:  11222741464018821716308593 75
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/operators.py
Enter the First Number: 67
Enter the Second Number: 79
Sum:  146
Subtraction:  -12
Multiplication:  5293
Division:  0.8481012658227848
Floor Division:  0
Modulo:  67
Power:  1819321338736902125229335845622505507704549101261646195761894037378497941 4
4547153562345267813354228846795668175133962580161207340795515193091 4603
>>>
Ln: 25  Col: 0
```

RESULT:

Thus, the python program using arithmetic operators is successfully completed.

Program No: 1 D File Name: Ex. No: Date: _____	PYTHON PROGRAM FOR DIVISIBLE NUMBERS BY 2 AND 3

AIM:

To write a Python Program to perform Divisible numbers by 2 and 3 of a given number.

ALGORITHM:

Step1: Start

Step2: Read the value for num.

Step3: Check if the last digit of the number is even.

Step4: If the number is even, then the number is divisible by 2.

Step5: Check if the sum of the digits of the number is divisible by 3.

Step6: If it is, then the number is divisible by 3.

Step7: Print the value of num.

Step8: Stop

PROGRAM:

```
num=8
print ("num = ",num)
if num%2==0:
  if num%3==0:
    print ("Divisible by 3 and 2")
  else:
    print ("divisible by 2 not divisible by 3")
else:
  if num%3==0:
    print ("divisible by 3 not divisible by 2")
  else:
    print ("not Divisible by 2 not divisible by 3")
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========================================= RESTART: C:/Users/Szenthil/AppData/Local
/Programs/Python/Python310/div by 2 and 3.py ====================================
===
num =  8
divisible by 2 not divisible by 3
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/div by 2 an
d 3.py
num =  15
divisible by 3 not divisible by 2
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/div by 2 an
d 3.py
num =  5
not Divisible by 2 not divisible by 3
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/div by 2 an
d 3.py
num =  12
Divisible by 3 and 2
>>>
                                                                    Ln: 14  Col: 37
```

RESULT:

Thus, the python program for perform Divisible numbers by 2 and 3 of a given number is successfully completed.

Program No: 1 E File Name: Ex. No: Date: _____	PYTHON PROGRAM TO COMPUTE THE GCD OF TWO NUMBERS

AIM:

To write a Python Program to compute the GCD of two numbers.

ALGORITHM:

Step1: Start

Step2: Read the value of first number a.

Step3: Read the Value of Second number b.

Step4: Calculate rem=a%b.

Step5: Repeat the steps 6 to 7 until rem!=0.

Step6: Assign a=b.

Step7: Assign b=rem.

Step8: Calculate rem=a%b.

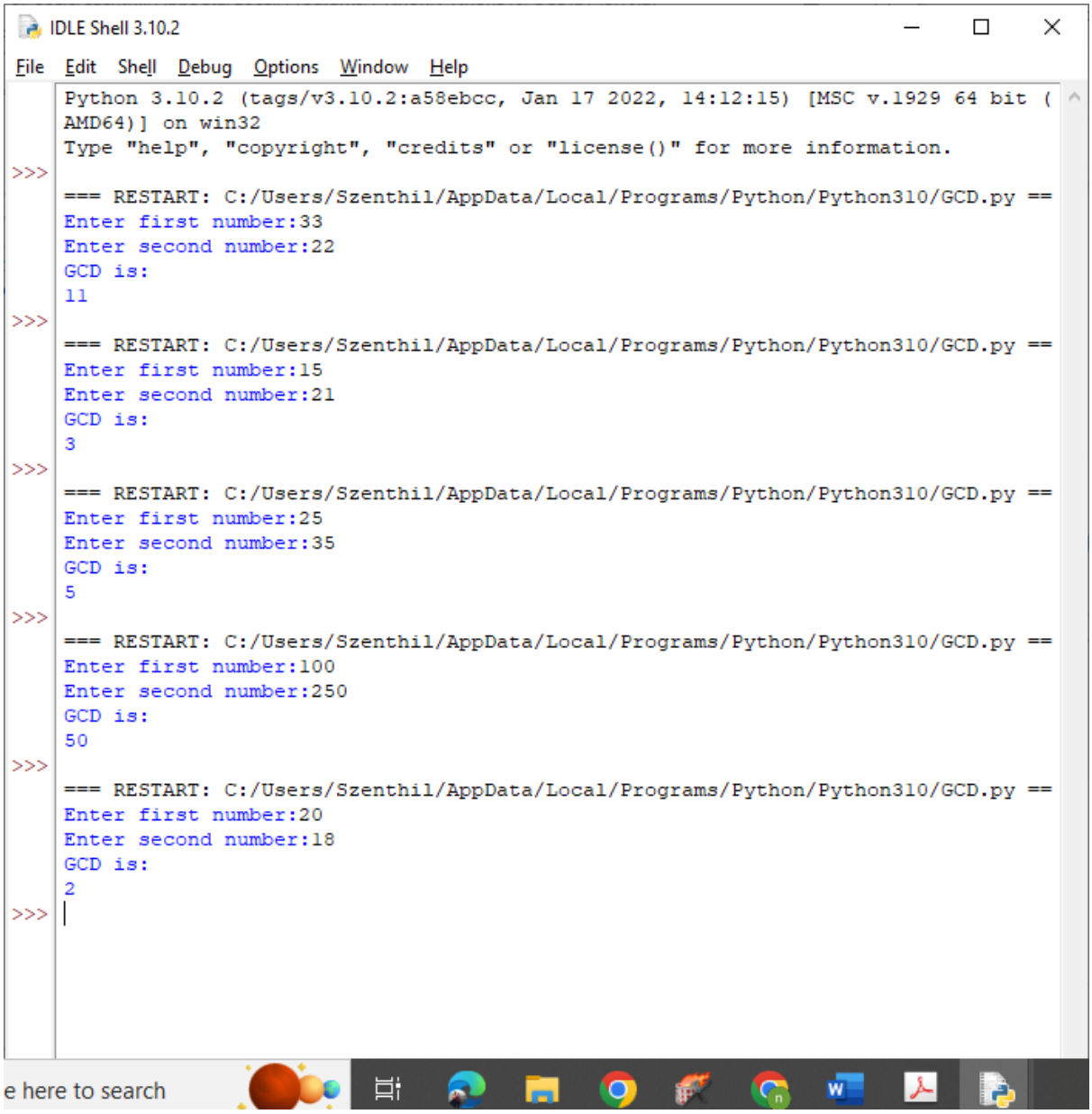
Step9: Print the GCD of given number is b.

Step10: Stop

PROGRAM:

```
def gcd(a,b):
    if(b==0):
        return a
    else:
        return gcd(b,a%b)
a=int(input("Enter first number:"))
b=int(input("Enter second number:"))
GCD=gcd(a,b)
print("GCD is: ")
print(GCD)
```

OUTPUT:



RESULT:

Thus, the python program for GCD is successfully completed.

Program No: 1 F File Name: Ex. No: Date: _____	STRINGS IN PYTHON

AIM:

To write a Python Program to compute the operations and functions of strings in python.

ALGORITHM:

Step1: Start

Step2: Read the value of text.

Step3: lower() – Converts all upper case characters in a string in to lowercase.

Step4: upper() – Converts all lower case characters in a string in to uppercase.

Step5: title() – Convert string to title case.

Step6: swapcase() - Swap the cases of all character in to a string.

Step7: capitalize() – Convert the first character of a string to uppercase.

Step8: Read the function.

Step9: Print the result.

Step10: Stop

PROGRAM:

```
text = “WelcoMe To PytHON”

# string to upper case

print("\nConverted String:")

print(text.upper())

 # string to lower case

print("\nConverted String:")

print(text.lower())
```

```
# title upper case and rest to lower case
print("\nConverted String:")
print(text.title())
# upper case character to lowercase and vice versa
print("\nConverted String:")
print(text.swapcase())
# convert the first character of a string to uppercase
print("\nConverted String:")
print(text.capitalize())
# original string never changes
print("\nOriginal String")
print(text)
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/strings.py

Converted String:
WELCOME TO PYTHON

Converted String:
welcome to python

Converted String:
Welcome To Python

Converted String:
wELCOmE tO pYThon

Converted String:
Welcome to python

Original String
WelcoMe To PytHON
>>>
Ln: 3  Col: 0
```

RESULT:

Thus, the python program to compute the operations and functions of strings in python has been successfully completed.

Program No: 2 A File Name: Ex. No: Date: _____	LISTS IN PYTHON

AIM:

To write a Python Program to compute the methods and functions of python lists.

ALGORITHM:

Step1: Start

Step2: Read the list value of a.

Step3: append() – Adds an element to the end of the list..

Step4: copy() – Returns the shallow copy of the list.

Step5: count() – Returns the number of times a specified element appears in the list.

Step6: extend() – Adds elements from another list to the end of the current list.

Step7: index() – Returns the index of the first occurrence of a specified element.

Step8: insert() – Inserts an element at a specified position.

Step9: pop() – Removes and returns the element at the specified position (or the last element if no index is specified).

Step10: remove() – Removes the first occurrence of a specified element.

Step11: reverse() – Reverses the order of the elements in the list.

Step12: sort() – Sorts the list in ascending order (by default).

Step13: clear() – Removes all elements from the list.

Step14: Read the function.

Step15: Print the result.

Step16: Stop

PROGRAM:

```
a = [1, 2, 3, 4, 5, 6]

# Add 7 to the end of the list

a.append(7)

print(a)

# Create a copy of the list

b = a.copy()

print(b)

# Count occurrences of 5 in the list

print(a.count(5))

# Extend list a by adding elements from list [8, 9]

a.extend([8, 9])

print(a)

# Find the index of 7 in the list

print(a.index(7))

# Insert 2 at index 1

a.insert(1, 2)

print(a)

# Remove and return the last element in the list

a.pop()

print(a)

# Remove the first occurrence of 2
a.remove(2)

print(a)
```

```
# Reverse the list order
a.reverse()
print(a)

# Sort the list in ascending order
a.sort()
print(a)

# Remove all elements from the list
a.clear()
print(a)
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
1
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
6
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
6
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 2, 3, 4, 5, 6, 7, 8, 9]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7, 8]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7, 8]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[8, 7, 6, 5, 4, 3, 2, 1]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[1, 2, 3, 4, 5, 6, 7, 8]
>>>
= RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/listsl.py =
[]
>>>
Ln: 39  Col: 0
```

RESULT:

Thus, the python program to compute the methods and functions of python list has been successfully completed.

Program No: 2 B File Name: Ex. No: Date: _____	TUPLES IN PYTHON

AIM:

To write a Python Program to compute the methods and functions of python tuples.

ALGORITHM:

Step1: Start

Step2: Read the tuple value of variable tuple.

Step3: length() – Returns the total length of the tuple.

Step4: Concatenation (+) – It allows for the creation of new tuples by combining the existing ones.

Step5: Repetition(*) – Returns multiple copies of the same tuple.

Step6: maximum() – Returns the largest element from the tuple.

Step7: minimum() – Returns the smallest element from the tuple.

Step8: index() – Accessing the specific element from the tuple by using its position or index number.

Step9: count() – Returns the number of times a specified value appears in the tuple.

Step10: delete() – Deletes the whole tuple.

Step11: Read the function.

Step12: Print the result.

Step13: Stop

PROGRAM:

```
tuple = (10, 20, 30, 40, 50, 60)
# Find the length of the tuple
print(len(tuple))
```

```
# Updating the tuples by using concatenation (+) operator
tuple1 = (10, 20, 30, 40, 50, 60)
tuple2 = (70, 80, 90)
tuple3 = tuple1+tuple2
print(tuple3)

# To make multiple copies of a tuple using repetition (*) operator
print(tuple*2)

# To find the maximum value in the given tuple using max() function
print(max(tuple))

# To find the minimum value in the given tuple using min() function
print(min(tuple))

# To access the specific element from the given tuple by using slice operator [:]
print(tuple.index(50))

# To find the count of specified element the given tuple using count() function
print(tuple.count(10))

# To delete the entire tuple by using del() function
del tuple
print(tuple)
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
6
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
(10, 20, 30, 40, 50, 60, 70, 80, 90)
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
(10, 20, 30, 40, 50, 60, 10, 20, 30, 40, 50, 60)
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
60
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
10
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
4
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
1
>>>
== RESTART: C:/Users/Szenthil/AppData/Local/Programs/Python/Python310/tuple.py =
<class 'tuple'>
>>>
Ln: 27  Col: 0
```

RESULT:

Thus, the python program to compute the methods and functions of python tuples has been successfully completed.

Program No: 2 C File Name: Ex. No: Date: _____	DICTIONARIES IN PYTHON

AIM:

To write a Python Program to compute the methods and functions of python dictionary.

ALGORITHM:

Step1: Start

Step2: Read the tuple value of variable a.

Step3: Creating dictionary – It consists of (Key: Value) pairs separates with coma (,) and encloses in curly brackets {}. Each key is separated from it value by a colon (:).

Step4: Accessing dictionary – To access dictionary we can use square brackets along with the key to obtain its value.

Step5: Adding in dictionary – To add dictionary we can use square brackets along with the key to obtain its value.

Step6: Updating dictionary – We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry.

Step7: Deletion in dictionary – We can remove individual dictionary elements or clear the entire contents of a dictionary by using clear() function.

Step8: length() – Returns the total length of the dictionary.

Step9: copy() – Allows to copy the key-value pairs in to another variable..

Step10: clear() – Delete the specified key-values and also delete entire dictionary.

Step11: pop() – Deletes an item from the dictionary and it will return the value of a key which is removed.

Step12: popitem() – Removes the recently inserted item. The value of removed key-value pair will be returned.

Step13: Read the function.

Step14: Print the result.

Step15: Stop

PROGRAM:

```
# Creating dictionary

dict = {'Name' : 'Szenthil', 'Roll no' : 123, 'Department' : 'MBA'}

print(dict)

# Accessing dictionary

print(dict['Name'])

print(dict['Roll no'])

print(dict['Department'])

# Adding dictionary

dict['Course'] = 'Business Analytics'

print(dict)

# Updating dictionary

dict['Course'] = 'Business Forecasting'

print(dict)

# Deleting dictionary

del dict['Department']

print(dict)

# Find the length of the dictionary

print(len(dict))

# Copy the key-value pairs in to another variable

dict1=dict.copy()

print(dict1)

# Remove all key-value pairs

dict.clear()
```

```
print(dict)

# Delete any item from the dictionary

dict.pop('Department')

print(dict)

# Delete recently added item from the dictionary

dict.popitem()

print(dict)
```

OUTPUT:

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD
64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123, 'Department': 'MBA'}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
Szenthil
123
MBA
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123, 'Department': 'MBA', 'Course': 'Business Analy
tics'}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123, 'Department': 'MBA', 'Course': 'Business Forec
asting'}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
3
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123, 'Department': 'MBA'}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123}
>>>
== RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\dicti.py =
{'Name': 'Szenthil', 'Roll no': 123}
>>>
Ln: 35  Col: 0
```

RESULT:

Thus, the python program to compute the methods and functions of python dictionary has been successfully completed.

Program No: 3 File Name: Ex. No: Date: _____	ILLUSTRATION ON LAMBDA AND FILTERS IN PYTHON

AIM:

To study and implement the use of **lambda functions** and **filter functions** in Python for concise and efficient data processing.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a list of elements (e.g., numbers).

Step 3: Create a lambda function to define the desired operation or condition.

Step 4: Apply the lambda function:

- Directly for simple calculations, or
- Inside the `filter()` function to select specific elements.

Step 5: Store the result in a variable.

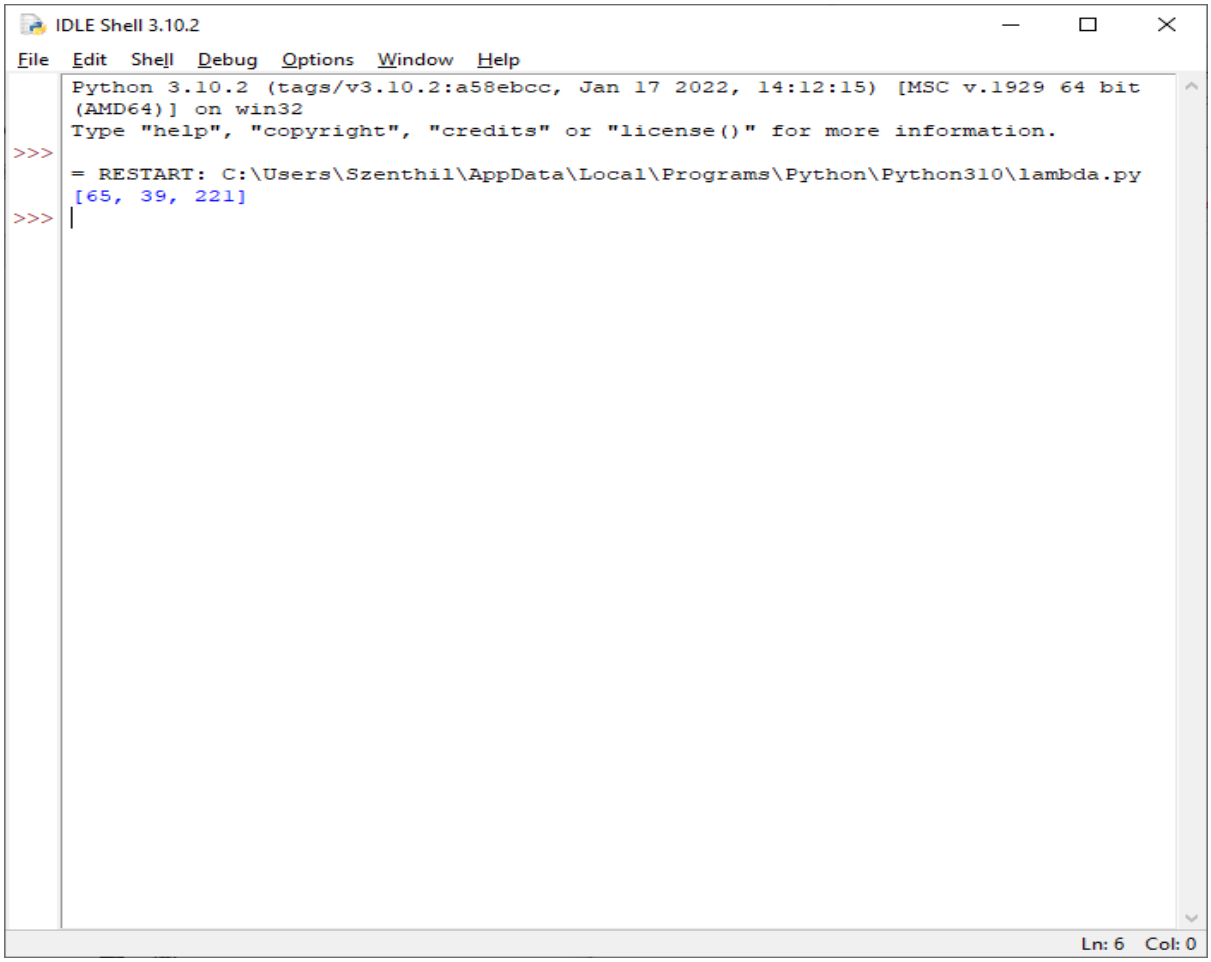
Step 6: Display the output.

Step 7: Stop the program.

PROGRAM:

```
a = [12, 65, 54, 39, 102, 339, 221, 50, 70]

# Using filter() with lambda to get numbers divisible by 13

res = list(filter(lambda x: (x % 13 == 0), a))


# Printing the result
print(res)
```

OUTPUT



RESULT:

Thus, the python program for implementing **lambda functions** and **filter functions** has been successfully completed.

Program No: 4 File Name: Ex. No: Date: _____	IMPLEMENTING INHERRITANCE

AIM:

To study and implement **single inheritance in Python**, where a child class inherits properties and methods from a parent class.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a parent class Animal.

Step 3: Create a constructor __init__ in Animal to initialize the attribute name.

Step 4: Define a method info() in the parent class to display the animal's name.

Step 5: Define a child class Dog that inherits from Animal.

Step 6: Create a constructor __init__ in Dog with parameters name and breed.

Step 7: Use super().__init__(name) to call the parent class constructor.

Step 8: Initialize the additional attribute breed in the child class.

Step 9: Define a method details() in Dog to display the dog's name and breed.

Step 10: Create an object of the Dog class.

Step 11: Call the parent class method info() using the object.

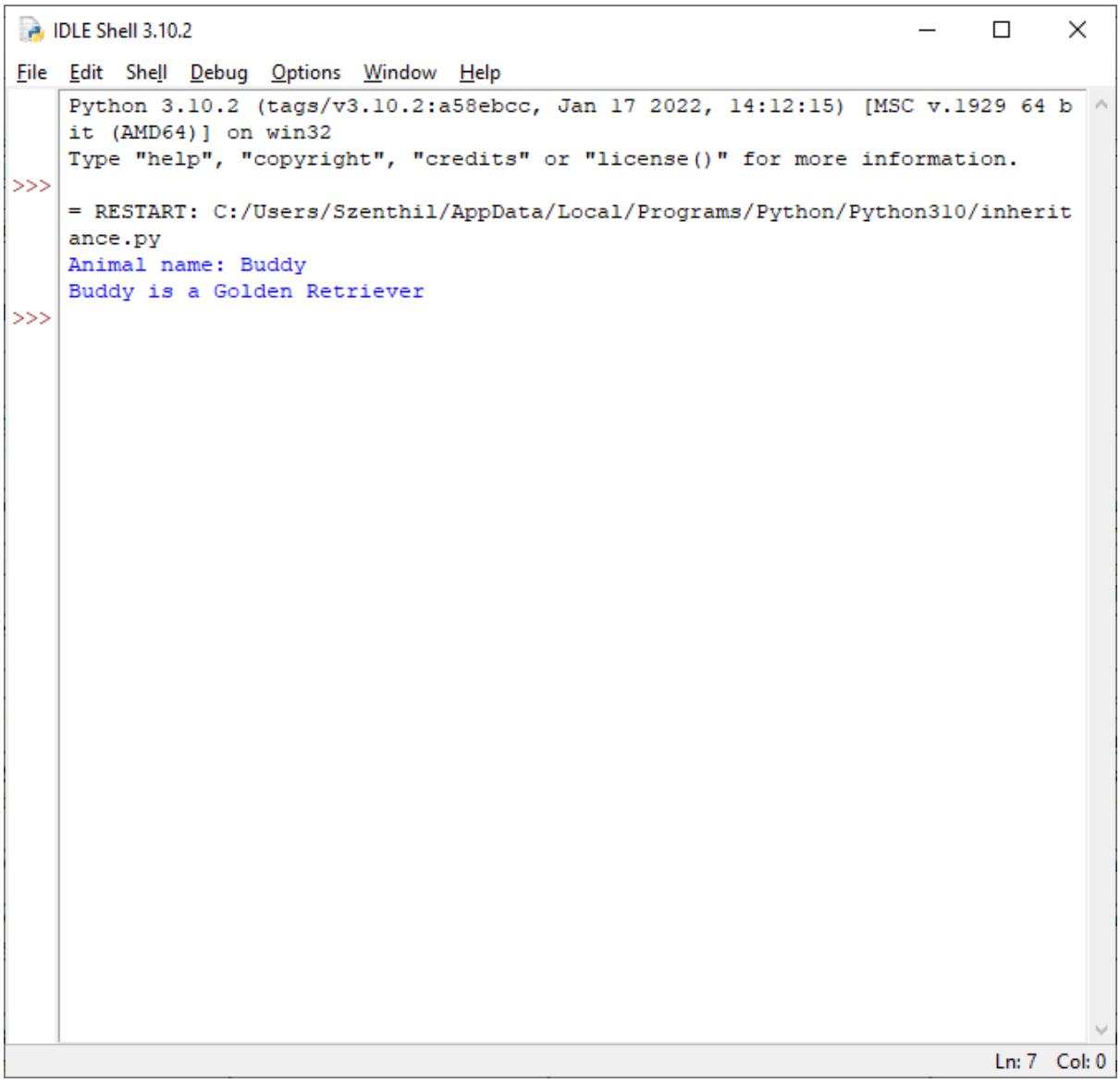
Step 12: Call the child class method details() using the object.

Step 13: Stop the program.

PROGRAM:

```
# Parent Class: Animal
class Animal:
    def __init__(self, name):
        self.name = name

    def info(self):
        print("Animal name:", self.name)

# Child Class: Dog
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # Call parent constructor
        self.breed = breed

    def details(self):
        print(self.name, "is a", self.breed)

d = Dog("Buddy", "Golden Retriever")
d.info()      # Parent method
d.details()   # Child method
```

OUTPUT



RESULT:

Thus, the python program to compute **single inheritance in Python**, where a child class inherits properties and methods from a parent class has been successfully completed.

Program No: 5 File Name: Ex. No: Date: _____	IMPLEMENTING METHOD OVERLOADING

AIM:

To study and implement **method overloading in Python** using variable-length arguments (*args), since Python does not support traditional method overloading.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a function `add(*args)` to accept a variable number of arguments.

Step 3: Initialize a variable `res` to 0.

Step 4: Iterate through each argument using a loop.

Step 5: Add each argument to `res`.

Step 6: Return or print the result.

Step 7: Call the function with different numbers of arguments:

- Two arguments
- Three arguments
- More arguments

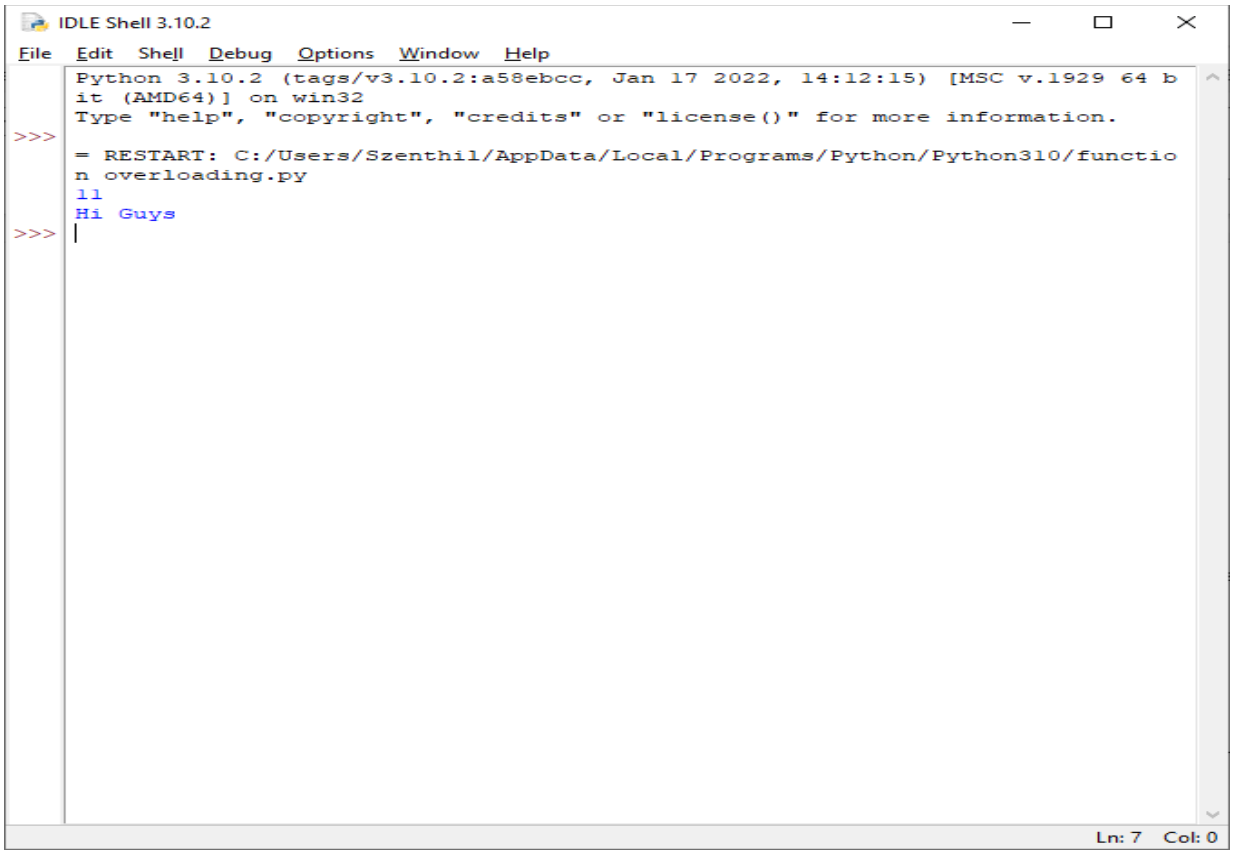
Step 8: Display the results.

Step 9: Stop the program.

PROGRAM:

```
def add(datatype, *args):
    if datatype == 'int':
        res = 0
    elif datatype == 'str':
        res = ''

    for item in args:
        res += item

    print(res)

add('int', 5, 6)
add('str', 'Hi ', 'Guys')
```

OUTPUT



RESULT:

Thus, the python program to import CSV data files using pandas has been successfully completed.

Program No: 6 File Name: Ex. No: Date: _____	ILLUSTRATION ON HOW TO RAISE AN EXCEPTION

AIM:

To study and implement **exception handling in Python** to handle runtime errors and ensure smooth program execution.

ALGORITHM:

Step 1: Start the program.

Step 2: Write the code that may produce an error inside a try block.

Step 3: Add one or more except blocks to handle specific exceptions.

Step 4: Optionally, include an else block to execute code when no exception occurs.

Step 5: Include a finally block to execute code that must run regardless of exceptions.

Step 6: Execute the program.

Step 7: Display appropriate output based on whether an exception occurred or not.

Step 8: Stop the program.

Step 9 :

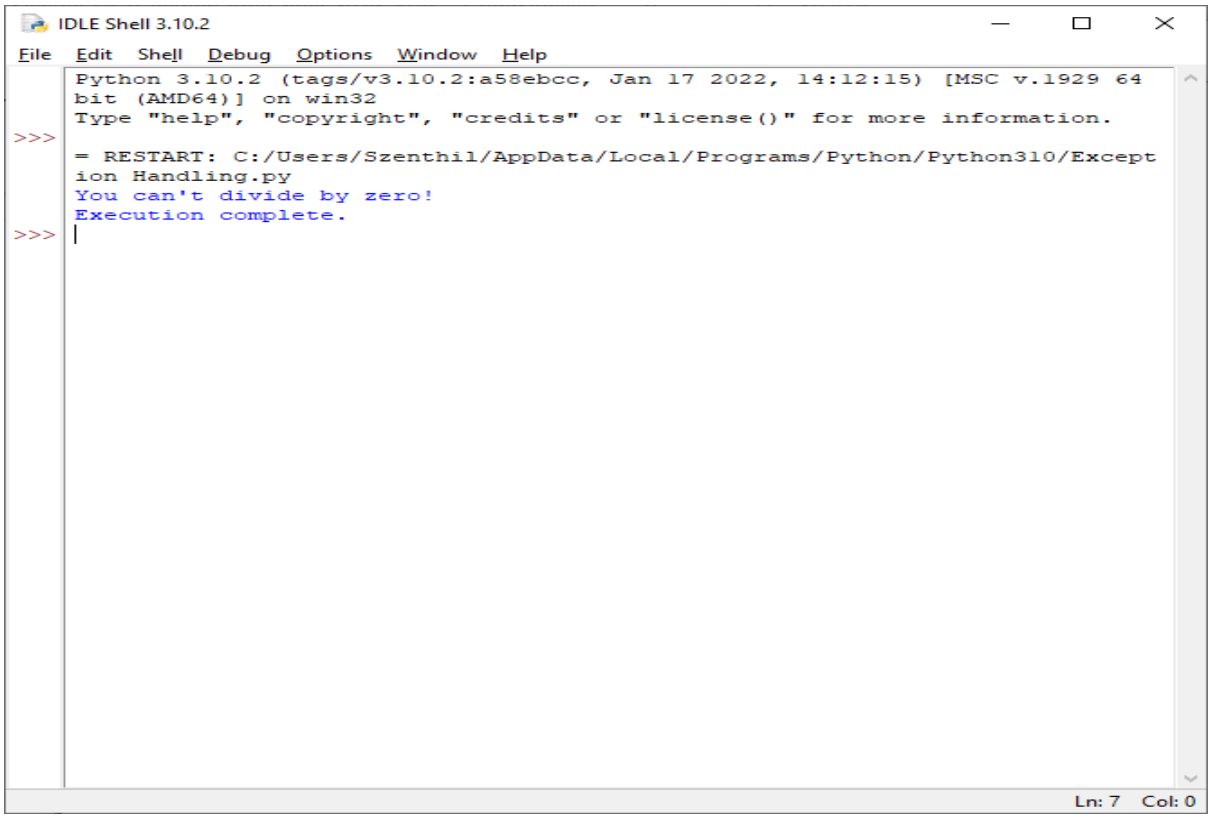
Explanation

- try: Contains risky code
- except: Handles errors
- else: Runs if no error occurs
- finally: Always executes

PROGRAM:

```
try:
    n = 0
    res = 100 / n

except ZeroDivisionError:
    print("You can't divide by zero!")

except ValueError:
    print("Enter a valid number!")

else:
    print("Result is", res)

finally:
    print("Execution complete.")
```

OUTPUT



RESULT:

Thus, the python program to compute the **exception handling in Python** to handle runtime errors has been successfully completed.

Program No: 7 File Name: Ex. No: Date: _____	IMPLEMENTING MODULES [CREATING AND IMPORTING]

AIM:

To create a Python module with user-defined functions and import it into another program to perform operations like addition, subtraction, and displaying greetings.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a module file named `mymodule.py`.

Step 3: Define functions inside the module:

- `add(a, b)` to perform addition
- `sub(a, b)` to perform subtraction
- `greet(name)` to return a greeting message

Step 4: Save the module file.

Step 5: Create the main program file `main.py`.

Step 6: Import the module using `import mymodule`.

Step 7: Call the functions from the module using:

- `mymodule.add()`
- `mymodule.sub()`
- `mymodule.greet()`

Step 8: Display the results.

Step 9: Stop the program.

PROGRAM:

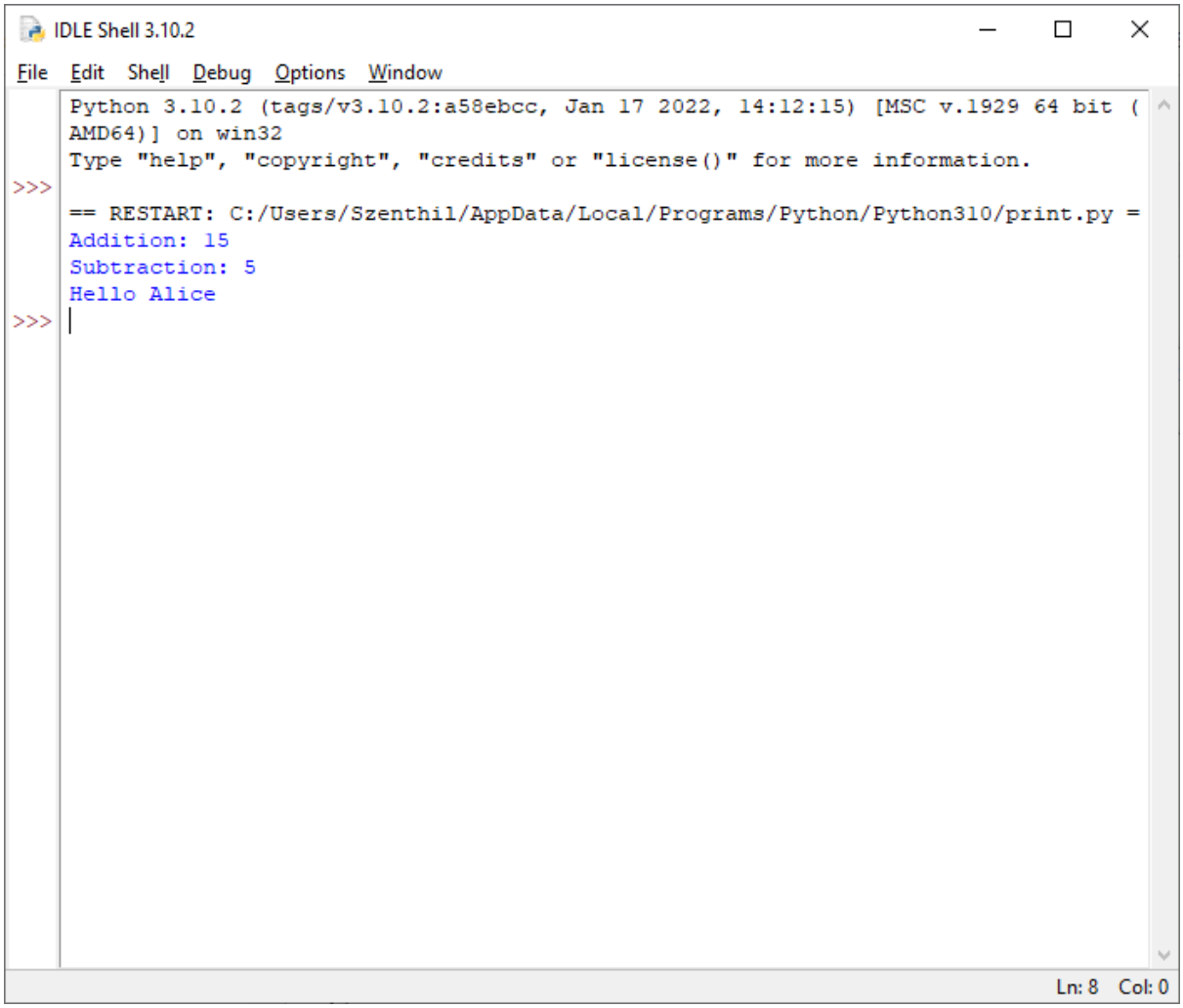
Create Module File (mymodule.py)

```
# mymodule.py

def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def greet(name):
    return "Hello " + name
```

Main Program (main.py)

```
# main.py

import mymodule

# Using functions from module
print("Addition:", mymodule.add(10, 5))
print("Subtraction:", mymodule.sub(10, 5))
print(mymodule.greet("Alice"))
```

OUTPUT:



RESULT:

Thus, the python program to compute Python module with user-defined functions and import it into another program has been successfully completed.

Program No: 8 File Name: Ex. No: Date: _____	IMPLEMENTING THREADS

AIM:

To study and implement **multithreading in Python with arguments**, where a thread executes a function with parameters and runs concurrently with the main program.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the required modules: `threading` and `time`.

Step 3: Define a function `my_function(argument1, argument2)` that:

- Prints the received arguments
- Simulates a task using `time.sleep()`
- Displays completion message

Step 4: Create a thread object using `threading.Thread()`.

Step 5: Pass the function as the `target` and arguments using the `args` parameter.

Step 6: Start the thread using the `start()` method.

Step 7: Execute statements in the main thread (parallel execution).

Step 8: Use `join()` to wait for the thread to finish execution.

Step 9: Display the final completion message.

Step 10: Stop the program.

PROGRAM:

```
import threading
import time

def my_function(argument1, argument2):
    print(f"Thread started with args: {argument1} and {argument2}")
    time.sleep(2) # Simulate some I/O work
    print("Thread finished its task.")

# Create and start the thread
my_thread = threading.Thread(target=my_function, args=('Hello', 'World',))
my_thread.start()

print("Main thread: Doing other work.")

# Wait for the thread to finish
my_thread.join()
print("Main thread: All threads are complete. Exiting program.")
```

OUTPUT

```
IDLE Shell 3.10.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Szenthil\AppData\Local\Programs\Python\Python310\threads.py
Main thread: Doing other work.Thread started with args: Hello and World

Thread finished its task.
Main thread: All threads are complete. Exiting program.
>>>
Ln: 9  Col: 0
```

RESULT:

Thus, the program to compute **multithreading in Python with arguments**, where a thread executes a function with parameters has been successfully completed.

Program No: 9 A File Name: Ex. No: Date: _____	ILLUSTRATION ON COMMAND LINE ARGUMENTS USING PYTHON

AIM:

To study and implement **command line arguments in Python** to calculate the sum of numbers passed during program execution.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the sys module.

Step 3: Access the list of command line arguments using sys.argv.

Step 4: Display:

- Total number of arguments using len(sys.argv)
- Script name using sys.argv[0]
- Input arguments using sys.argv[1:]

Step 5: Initialize a variable total = 0.

Step 6: Use a loop to iterate through each argument in sys.argv[1:].

Step 7: Convert each argument to integer using int().

Step 8: Add each value to total.

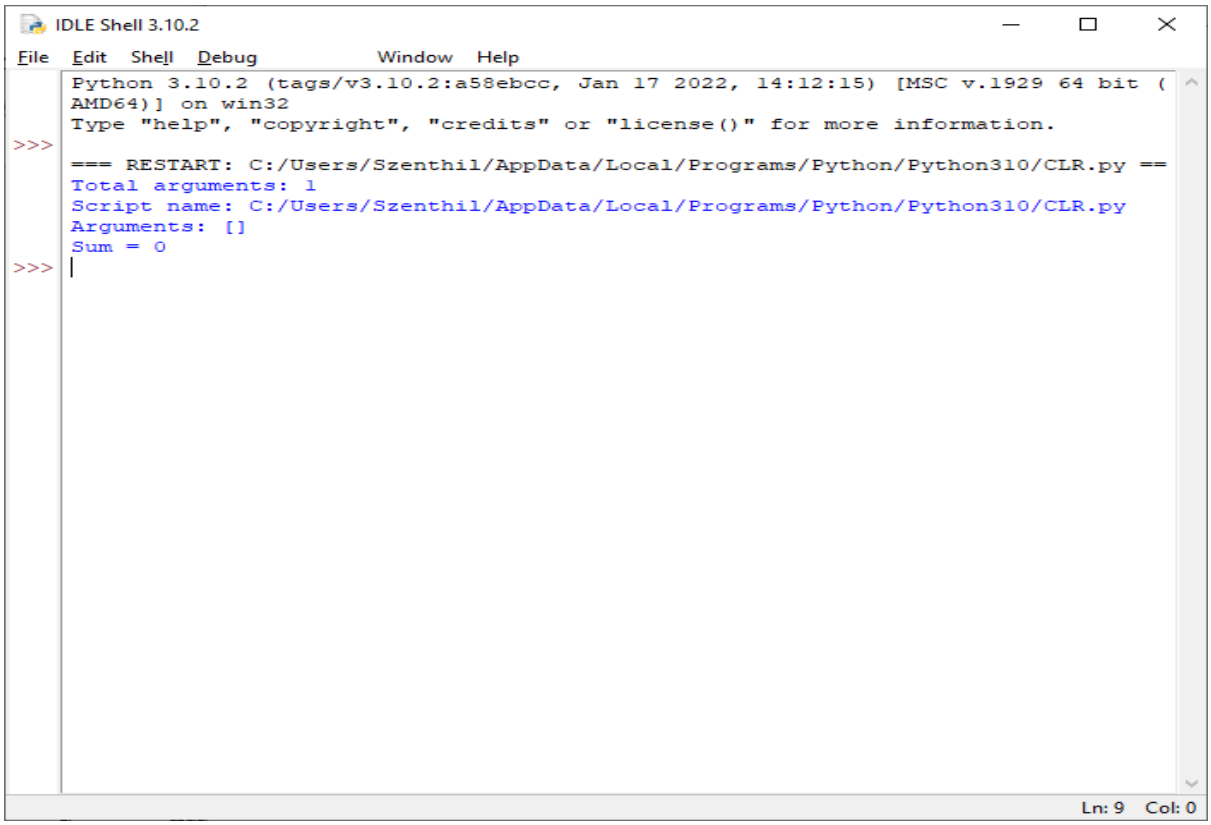
Step 9: Display the sum.

Step 10: Stop the program.

PROGRAM:

```
import sys

print("Total arguments:", len(sys.argv))
print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])

total = 0

for arg in sys.argv[1:]:

    total += int(arg)

print("Sum =", total)
```

OUTPUT:



RESULT:

Thus, the program to compute **command line arguments in Python** to calculate the sum of numbers passed during program execution has been successfully completed.

Program No: 9 B File Name: Ex. No: Date: _____	ILLUSTRATION ON REGULAR EXPRESSIONS IN PYTHON

AIM:

To study and implement **regular expressions in Python** for pattern matching and text processing.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the `re` module.

Step 3: Define a string on which operations will be performed.

Step 4: Specify a pattern (word or sequence to search).

Step 5: Use a RegEx function such as:

- `re.search()` → finds first match
- `re.match()` → checks match at beginning
- `re.findall()` → returns all matches

Step 6: Store the result in a variable.

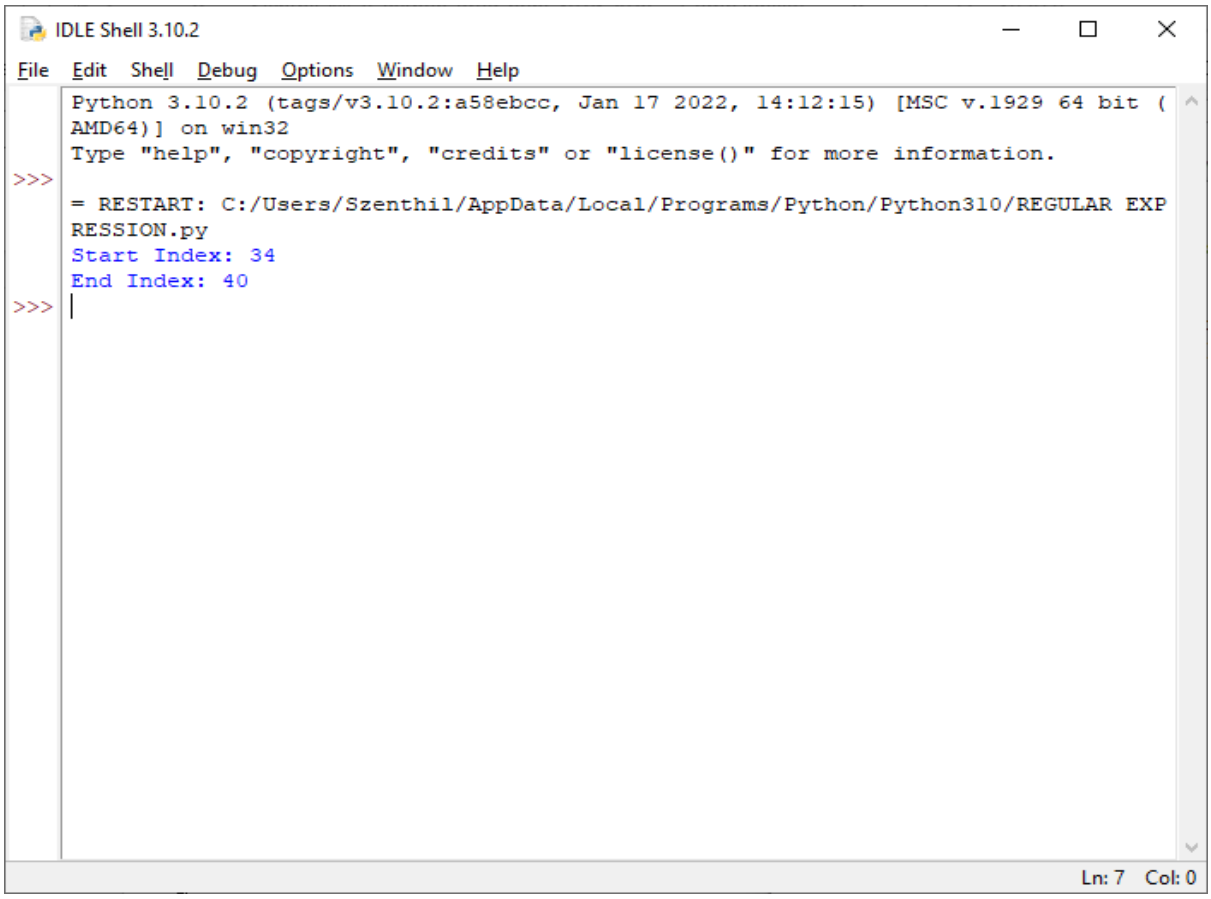
Step 7: Display the result.

Step 8: Stop the program.

PROGRAM:

```
import re

s = 'GeeksforGeeks: A computer science portal for geeks'
match = re.search(r'portal', s)

print('Start Index:', match.start())
print('End Index:', match.end())
```

OUTPUT:



RESULT:

Thus, the program for **regular expressions in Python** was successfully executed.

Program No: 10 File Name: Ex. No: Date: _____	DESCRIPTIVE STATISTICS USING NUMPY

AIM:

To study and implement **statistical measures in Python** using NumPy and SciPy, including measures of central tendency and dispersion.

ALGORITHM:

Step 1: Start the program.

Step 2: Import required libraries:

- numpy for numerical operations
- scipy.stats for mode calculation

Step 3: Define a dataset using a NumPy array.

Step 4: Calculate **measures of central tendency**:

- Mean using np.mean()
- Median using np.median()
- Mode using stats.mode()

Step 5: Calculate **measures of dispersion**:

- Minimum using np.min()
- Maximum using np.max()
- Range using np.ptp()
- Variance using np.var()
- Standard deviation using np.std()

Step 6: Calculate **quartiles**:

- First quartile (Q1) using np.percentile(data, 25)
- Third quartile (Q3) using np.percentile(data, 75)

Step 7: Compute **Interquartile Range (IQR)**:

- IQR = Q3 - Q1

Step 8: Display all calculated values.

Step 9: Stop the program.

PROGRAM:

```
import numpy as np
from scipy import stats # Required for the mode calculation

# Sample data
data = np.array([1, 2, 2, 3, 4, 5, 5, 5, 6, 7])

# Measures of Central Tendency
mean_value = np.mean(data)
median_value = np.median(data)
# Mode requires SciPy
mode_result = stats.mode(data)
mode_value = mode_result.mode[0]

# Measures of Dispersion
min_value = np.min(data)
max_value = np.max(data)
range_value = np.ptp(data) # Peak-to-peak range
variance_value = np.var(data)
std_deviation_value = np.std(data)
q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1

print(f"Mean: {mean_value}")
print(f"Median: {median_value}")
print(f"Mode: {mode_value}")
print(f"Minimum: {min_value}")
```

```
print(f"Maximum: {max_value}")
print(f"Range: {range_value}")
print(f"Variance: {variance_value}")
print(f"Standard Deviation: {std_deviation_value}")
print(f"Interquartile Range (IQR): {iqr}")
```

OUTPUT:

data = [1, 2, 2, 3, 4, 5, 5, 5, 6, 7]

Mean: 4.0

Median: 4.5

Mode: 5

Minimum: 1

Maximum: 7

Range: 6

Variance: 3.4

Standard Deviation: 1.8439088914585775

Interquartile Range (IQR): 3.0

RESULT:

Thus, the program for calculating **statistical measures (mean, median, mode, variance, standard deviation, and IQR)** using Python was successfully executed.

Program No: 11 File Name: Ex. No: Date: _____	ILLUSTRATE INDEXING OPERATIONS IN DATA FRAMES

AIM:

To study and illustrate **indexing operations in Pandas DataFrame** for accessing and manipulating data.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the pandas library.

Step 3: Create a DataFrame with sample data.

Step 4: Perform indexing operations:

- Select a column using df['column_name']
- Select multiple columns using df[['col1', 'col2']]
- Select rows using .loc[] (label-based indexing)
- Select rows using .iloc[] (position-based indexing)

Step 5: Display the selected data.

Step 6: Stop the program.

PROGRAM:

```
import pandas as pd

data = {'Name': ['John', 'Anna', 'Peter', 'Linda'],
        'Age': [28, 24, 35, 32],
        'City': ['New York', 'Paris', 'Berlin', 'London']}
df = pd.DataFrame(data, index=['row1', 'row2', 'row3', 'row4'])
print(df)
```

OUTPUT

	Name	Age	City
row1	John	28	New York
row2	Anna	24	Paris
row3	Peter	35	Berlin
row4	Linda	32	London

RESULT:

Thus, indexing operations in a **Pandas DataFrame** were successfully illustrated.

Program No: 12 File Name: Ex. No: Date: _____	ILLUSTRATE VARIOUS PLOTS USING PANDAS AND MATPLOTLIB

AIM:

To study and illustrate **various plots using Pandas and Matplotlib** for data visualization.

ALGORITHM:

Step 1:

Start the program.

Step 2:

Import required libraries:

- `pandas`
- `matplotlib.pyplot`

Step 3:

Create a dataset using a dictionary.

Step 4:

Convert the dataset into a Pandas DataFrame.

Step 5:

Generate different types of plots:

- Line plot
- Bar plot
- Histogram
- Scatter plot

Step 6:

Display each plot using `plt.show()`.

Step 7:

Stop the program.

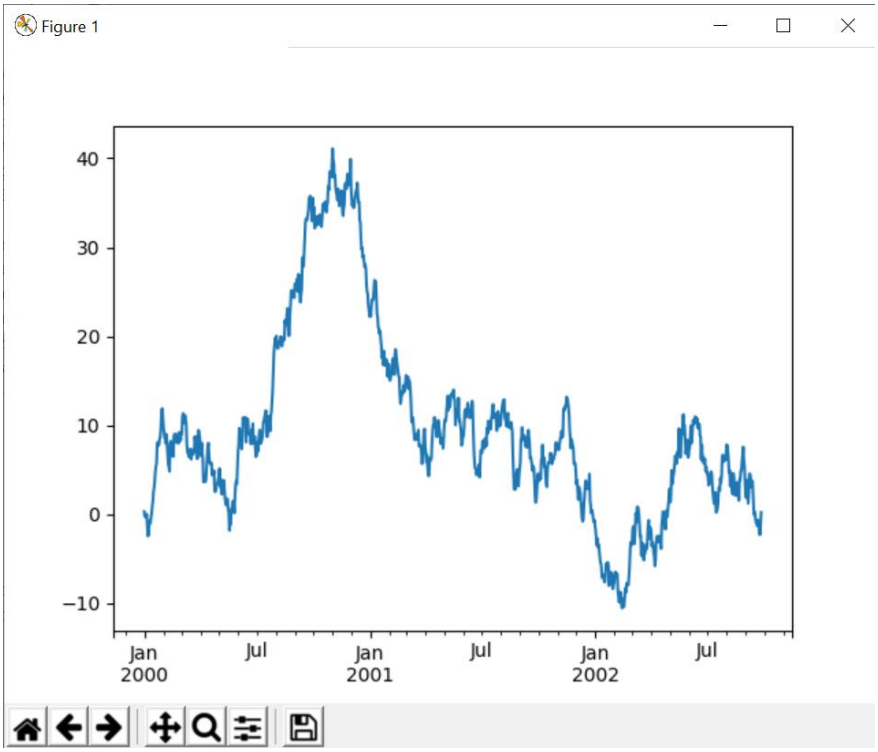
PROGRAM:

pip install matplotlib
pip install pandas
pip install numpy

Basic plotting

```
# import libraries

import matplotlib.pyplot as plt

import pandas as pd

import numpy as np

ts = pd.Series(np.random.randn(1000), index = pd.date_range('1/1/2000', periods = 1000))

ts = ts.cumsum()

ts.plot()

plt.show()
```

OUTPUT

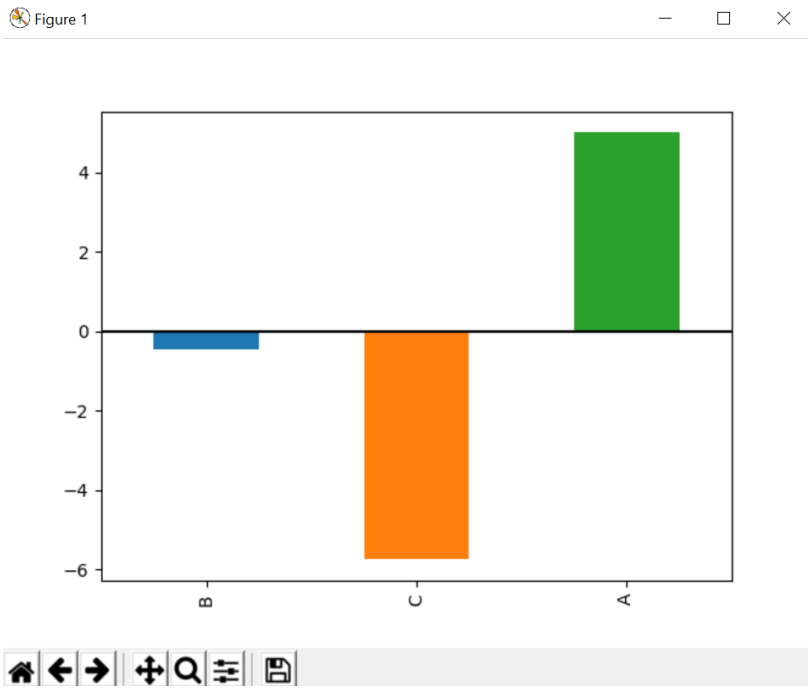


BAR PLOT

PROGRAM:

```
# importing libraries
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
ts = pd.Series(np.random.randn(1000), index = pd.date_range(
                        '1/1/2000', periods = 1000))
df = pd.DataFrame(np.random.randn(1000, 4), index = ts.index,
                        columns = list('ABCD'))
df3 = pd.DataFrame(np.random.randn(1000, 2),
            columns =['B', 'C']).cumsum()
df3['A'] = pd.Series(list(range(len(df))))
df3.iloc[5].plot.bar()
plt.axhline(0, color ='k')
plt.show()
```

OUTPUT

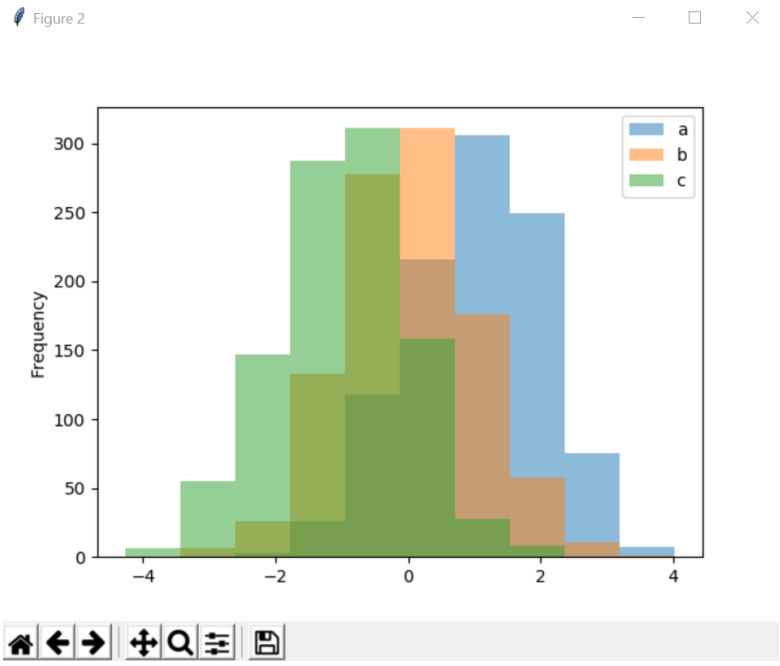


HISTOGRAMS

PROGRAM:

```
# importing libraries

import matplotlib.pyplot as plt

import pandas as pd

import numpy as np


df4 = pd.DataFrame({'a': np.random.randn(1000) + 1,

                    'b': np.random.randn(1000),

                    'c': np.random.randn(1000) - 1},

                         columns =['a', 'b', 'c'])

plt.figure()


df4.plot.hist(alpha = 0.5)

plt.show()
```

OUTPUT

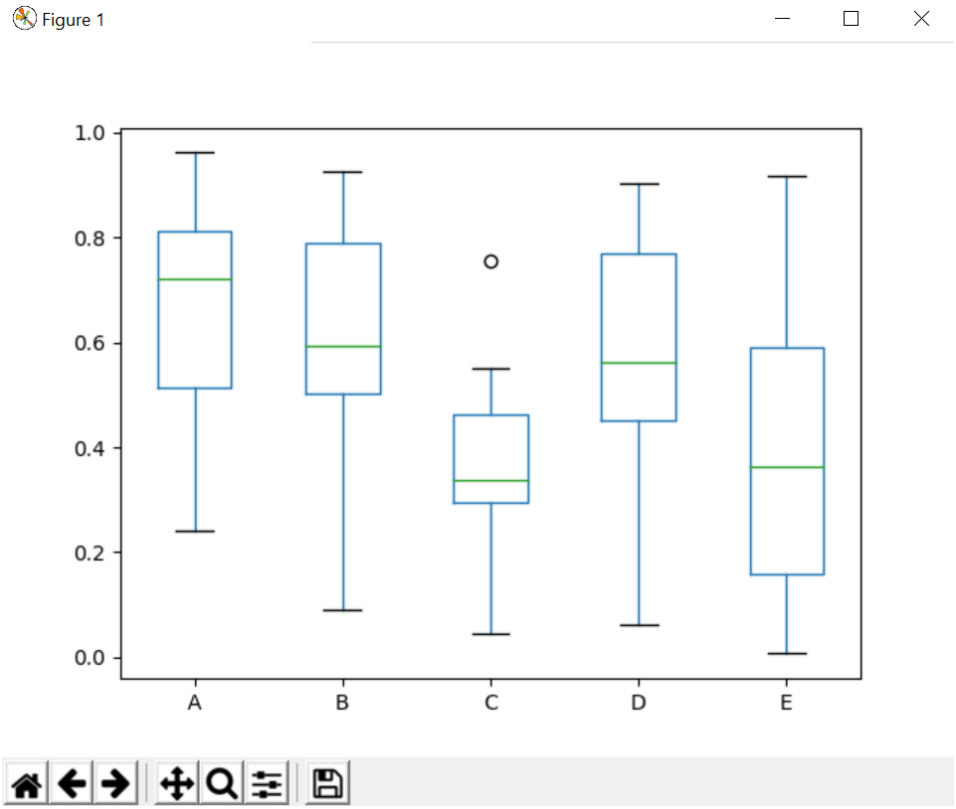


Box plot using Series and matplotlib

PROGRAM:

```
# importing libraries

import matplotlib.pyplot as plt

import pandas as pd

import numpy as np


df = pd.DataFrame(np.random.rand(10, 5),

      columns =['A', 'B', 'C', 'D', 'E'])


df.plot.box()

plt.show()
```

OUTPUT

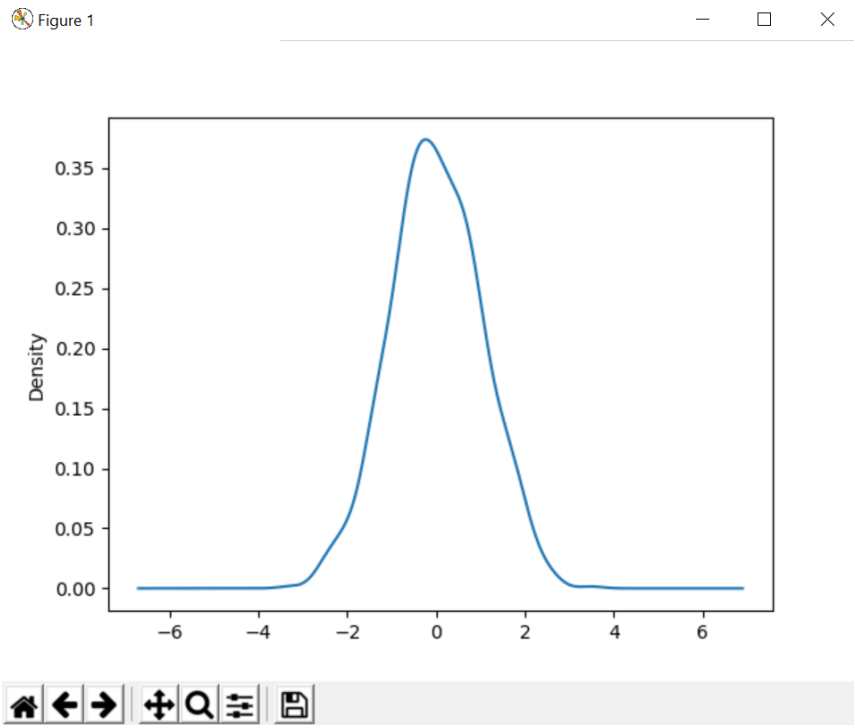


Density plot:

PROGRAM:

```
# importing libraries

import matplotlib.pyplot as plt

import pandas as pd

import numpy as np


df = pd.DataFrame(np.random.rand(10, 5),

      columns =['A', 'B', 'C', 'D', 'E'])


ser = pd.Series(np.random.randn(1000))

ser.plot.kde()

plt.show()
```

OUTPUT

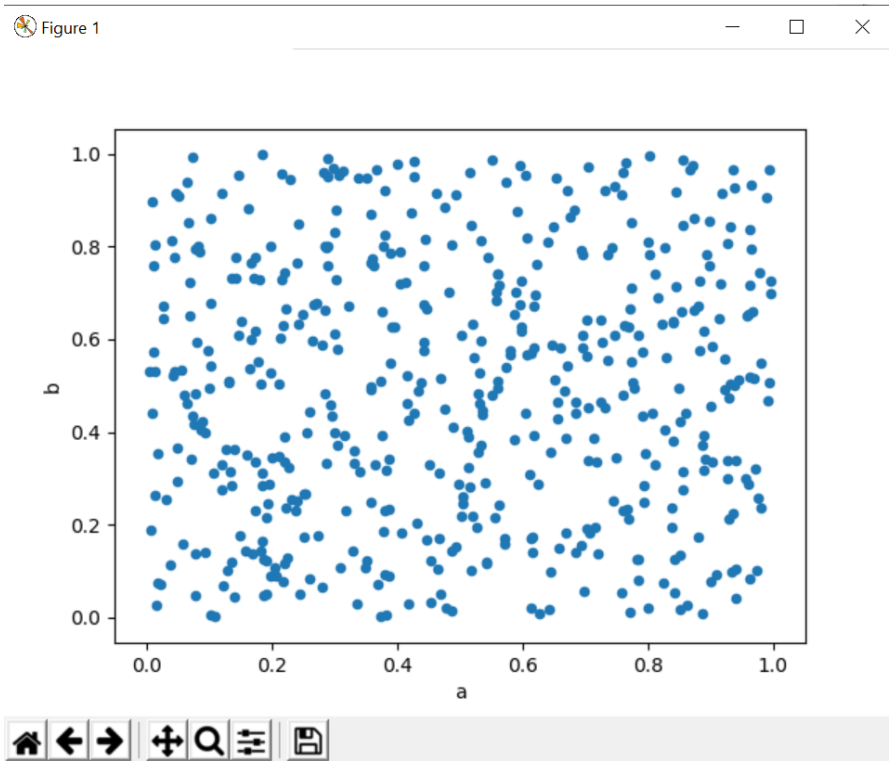


Scatter plot:

PROGRAM:

```
# importing libraries

import matplotlib.pyplot as plt

import pandas as pd

import numpy as np


df = pd.DataFrame(np.random.rand(500, 4),

          columns =['a', 'b', 'c', 'd'])


df.plot.scatter(x ='a', y ='b')

plt.show()
```

OUTPUT

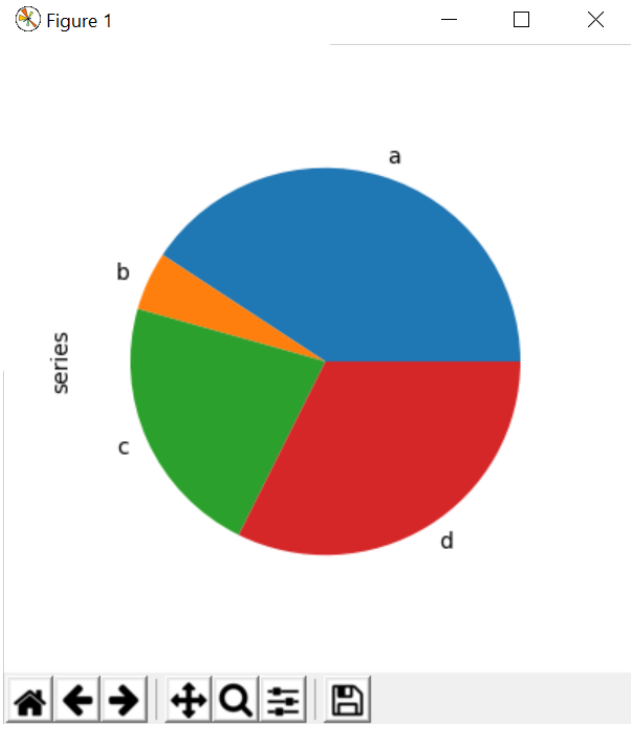


Pie plot:

PROGRAM:

```
# importing libraries
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

series = pd.Series(3 * np.random.rand(4),
  index =['a', 'b', 'c', 'd'], name ='series')

series.plot.pie(figsize =(4, 4))
plt.show()
```

OUTPUT



RESULT:

Thus, various plots using **Pandas and Matplotlib** were successfully illustrated.

Program No: 13 File Name: Ex. No: Date: _____	BUILDING GUI APPLICATIONS WITH TKINTER

AIM:

To study and implement a **basic GUI window using Tkinter** in Python.

ALGORITHM:

Step 1: Start the program.

Step 2:

Import the `tkinter` module.

Step 3:

Create the main window using `Tk()`.

Step 4:

Set the title of the window using `title()`.

Step 5:

Set the window size using `geometry()`.

Step 6:

Keep the window running using `mainloop()`.

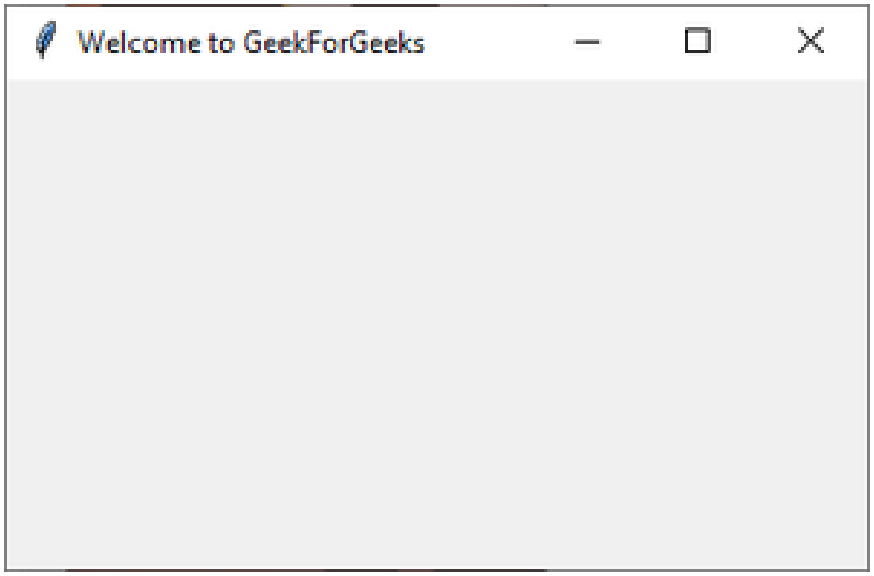
Step 7:

Stop the program when the window is closed.

PROGRAM:

```
# Import Module
from tkinter import *

# create root window
root = Tk()
# root window title and dimension
root.title("Welcome to GeekForGeeks")
# Set geometry (widthxheight)
root.geometry('350x200')
# all widgets will be here
# Execute Tkinter
root.mainloop()
```

OUTPUT



RESULT:

Thus, a **basic Tkinter GUI window** was successfully created and executed.

Program No: 14 File Name: Ex. No: Date: _____	CREATING TABLES USING SQLITE

AIM:

To study and implement **creating tables in SQLite using Python**.

ALGORITHM:

Step 1:

Start the program.

Step 2:

Import the `sqlite3` module.

Step 3:

Create or connect to a database using `connect()`.

Step 4:

Create a cursor object using `cursor()`.

Step 5:

Write an SQL query to create a table.

Step 6:

Execute the query using `execute()`.

Step 7:

Commit the changes using `commit()`.

Step 8:

Close the connection using `close()`.

Step 9:

Stop the program.

PROGRAM:

```
import sqlite3

# Step 1: Connect to database

conn = sqlite3.connect("school.db")

# Step 2: Create cursor

cur = conn.cursor()

# Step 3: Create table

cur.execute("""

CREATE TABLE IF NOT EXISTS student (

    id INTEGER PRIMARY KEY,

    name TEXT NOT NULL,

    marks INTEGER

)

""")

# Step 4: Insert records (ignore if already exists)

students = [

    (1, "John", 85),

    (2, "Alice", 90),
```

```
    (3, "Bob", 78)

]

cur.executemany("INSERT OR IGNORE INTO student VALUES (?, ?, ?)", students)

# Step 5: Commit changes

conn.commit()

# Step 6: Fetch and display data

print("Student Table Data:")

cur.execute("SELECT * FROM student")

for record in cur.fetchall():

    print("ID:", record[0], "| Name:", record[1], "| Marks:", record[2])

# Step 7: Close connection

conn.close()

print("Table created and data displayed successfully!")
```

OUTPUT

```
==================== RESTART: C:/Users/stu
Student Table Data:
ID: 1 | Name: John | Marks: 85
ID: 2 | Name: Alice | Marks: 90
ID: 3 | Name: Bob | Marks: 78
Table created and data displayed successfully!
```

RESULT:

Thus, the program for **creating a table using SQLite in Python** was successfully executed.

Program No: 15 File Name: Ex. No: Date: _____	ILLUSTRATION ON DATABASE CONNECTIVITY

AIM:

To demonstrate database connectivity using SQLite in Python and perform basic operations like creating a table and retrieving data.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the `sqlite3` module.

Step 3: Establish a connection to the database using `connect()`.

Step 4: Create a cursor object using `cursor()`.

Step 5:

Execute SQL queries using `execute()`:

- Create table
- Insert data
- Retrieve data

Step 6:

Fetch data using `fetchall()` or `fetchone()`.

Step 7:

Display the results.

Step 8:

Commit changes using `commit()`.

Step 9:

Close the connection using `close()`.

Step 10:

Stop the program.

PROGRAM:

```
import sqlite3

# Step 3: Connect to database

conn = sqlite3.connect("demo.db")

# Step 4: Create cursor

cursor = conn.cursor()

# Step 5: Create table and insert data

cursor.execute("""

CREATE TABLE IF NOT EXISTS employee (

    emp_id INTEGER PRIMARY KEY,

    emp_name TEXT NOT NULL,

    department TEXT,

    salary REAL

)

""")

employees = [

    (1, "John Doe", "HR", 50000),

    (2, "Alice Smith", "IT", 60000),
```

```
    (3, "Bob Johnson", "Sales", 55000)

]

cursor.executemany("INSERT OR IGNORE INTO employee VALUES (?, ?, ?, ?)",
employees)

# Step 6: Retrieve and display data

cursor.execute("SELECT * FROM employee")

rows = cursor.fetchall()

print("Employee Table Data:")

for row in rows:

    print(f"ID: {row[0]} | Name: {row[1]} | Dept: {row[2]} | Salary: {row[3]}")

# Step 7: Commit and close connection

conn.commit()

conn.close()
```

OUTPUT

```
>>>
========== RESTART: C:/Users/student/Desktop/ex15.py ========
Employee Table Data:
ID: 1 | Name: John Doe | Dept: HR | Salary: 50000.0
ID: 2 | Name: Alice Smith | Dept: IT | Salary: 60000.0
ID: 3 | Name: Bob Johnson | Dept: Sales | Salary: 55000.0
>>>
```

RESULT:

Thus, the program for **database connectivity using SQLite in Python** was successfully implemented and executed.